



BÀI TẬP THỰC HÀNH KỸ THUẬT LẬP TRÌNH

BIÊN SOẠN: HUỲNH THÀNH LỘC

MỤC LỤC

BUỔI 01 KỸ THUẬT ĐỌC GHI FILE VĂN BẢN	2
BUỔI 02 - 03 KỸ THUẬT ĐỀ QUY	5
BUỔI 04 - 05 OOP: TÍNH ĐÓNG GÓI	9
BUỔI 06 OOP: TÍNH KẾ THỪA VÀ ĐA HÌNH (PHẦN 1).....	14
BUỔI 07 OOP: TÍNH KẾ THỪA VÀ ĐA HÌNH (PHẦN 2).....	17
BUỔI 08 OOP: LỚP TRỪU TƯỢNG.....	18

BUỔI 01

KỸ THUẬT ĐỌC GHI FILE VĂN BẢN

MỤC TIÊU

- Ôn tập kiến thức lập trình cơ bản
- Có kỹ năng đọc, ghi **File văn bản**

Bài tập 1.

Viết chương trình đọc ma một ma trận vuông $a[n \times n]$, ($1 \leq n \leq 100$) chứa các số nguyên từ file "*Input.txt*".

Input.txt
3
5 1 3
4 7 2
4 2 6

Định nghĩa các hàm chức năng sau:

- a) In các **phần tử nằm trên đường chéo chính** vào file "*DuongCheoChinh.txt*"

DuongCheoChinh.txt
5 7 6

- b) In ra màn hình các **phần tử nằm trên đường chéo phụ** vào file "*DuongCheoPhu.txt*"

DuongCheoPhu.txt
3 7 4

- c) In ra màn hình các **phần tử thuộc tam giác trên đường chéo chính** vào file "*TamGiacTren.txt*"

TamGiacTren.txt
5 1 3
7 2
6

Bài tập 2. Đếm ký tự

Cho chuỗi s (có độ dài nhỏ hơn 10^6), chỉ gồm các ký tự từ 'a' đến 'z', 'A' đến 'Z' và khoảng trắng. Viết hàm thực hiện các chức năng sau:

- **Đếm số lần xuất hiện** của các ký tự khác nhau (không tính khoảng trắng, không phân biệt chữ in HOA và chữ thường).
- Cho biết **ký tự nào xuất hiện nhiều lần nhất** và **số lần xuất hiện là bao nhiêu**? Nếu có nhiều ký tự khác nhau có cùng số lượng xuất hiện nhiều nhất, **chọn ký tự nhỏ nhất** (theo thứ tự từ điển).

Input

- Dòng duy nhất chứa chuỗi *s*

Output

- Các dòng đầu tiên, mỗi dòng cho biết ký tự (in HOA) và số lần xuất hiện tương ứng (xuất theo thứ tự từ điển).
- Dòng cuối cùng, ký tự xuất hiện nhiều nhất và số lần xuất hiện (nếu có nhiều ký tự cùng số lần xuất hiện nhiều nhất, chọn ký tự nhỏ nhất theo thứ tự từ điển).

Ví dụ

INPUT	OUTPUT
<i>Khoa Công nghệ thông tin HUFLIT</i>	A : 1 C : 1 E : 1 F : 1 G : 3 H : 4 I : 2 K : 1 L : 1 N : 4 O : 3 T : 3 U : 1 First Max Freq: H,4

Hướng dẫn: Dùng mảng để đếm số lần xuất hiện của từng ký tự.

Bài tập 3. Kiểm tra Password

Viết chương trình đọc vào 1 file "*Password.txt*" chứa các chuỗi *s* là các mật khẩu. Hãy kiểm tra mỗi mật khẩu có thỏa các tất cả các ràng buộc sau hay không?

- Có ít nhất 8 ký tự
- Phải chứa ký tự thường, ký tự hoa và ký tự số

Chương trình cần đảm bảo được những yêu cầu sau:

- Yêu cầu người dùng nhập vào đường dẫn của thư mục chứa file "*Password.txt*".
- Mở file "*Password.txt*" theo đường dẫn trên để đọc dữ liệu? Nếu **file không tồn tại** thông báo lỗi "*File not found!*" và yêu cầu nhập lại đường dẫn khác.
- Khi đọc file thành công, tiến hành kiểm tra các mật khẩu và ghi kết quả vào file "*PasswordValidation.txt*" đặt trong cùng thư mục với file "*Password.txt*", dưới dạng:
Password : Yes/No (Yes: hợp lệ, No: không hợp lệ)
- Đóng các luồng đọc, ghi file đang mở.

Ví dụ

Password.txt	PasswordValidation.txt
abc123456	abc123456 : No
123456789	123456789 : No
Abcd12	Abcd12 : No
Abcd123456	Abcd123456 : Yes

Bài tập 4. Tách họ tên

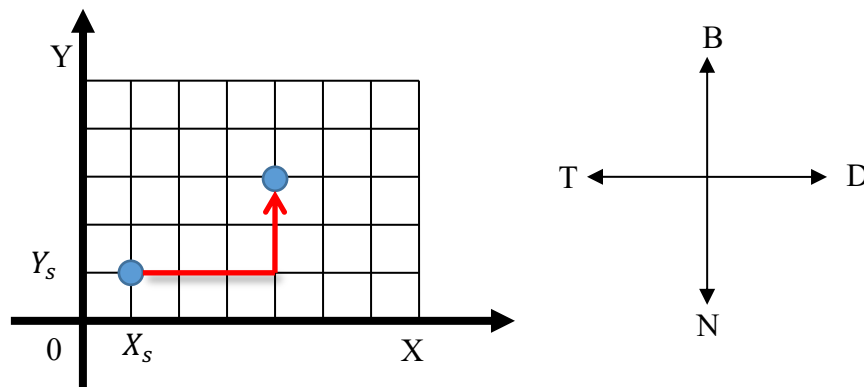
Viết chương trình đọc vào 1 file “*Names.txt*” chứa các chuỗi Họ Tên (các từ cách nhau bởi 1 ký tự khoảng trắng). Hãy viết chương trình **tách chuỗi Họ Tên thành 3 phần: Họ, Tên lót và Tên** và ghi kết quả vào file *SplitNames.txt*

Ví dụ

Names.txt	SplitNames.txt
Trần Thành Công	Trần Thành Công
Nguyễn Ngọc Quỳnh Hoa	- Họ: Trần
	- Tên lót: Thành
	- Tên: Công
	Nguyễn Ngọc Quỳnh Hoa
	- Họ: Nguyễn
	- Tên lót: Ngọc Quỳnh
	- Tên: Hoa

Bài tập 5.**Robot đi theo hướng đông, tây, nam, bắc**

Một Robot xuất phát từ điểm có tọa độ (X_s, Y_s) trong mặt phẳng tọa độ nguyên OXY , mỗi lần Robot đi được một đơn vị độ dài theo một trong 4 hướng: Đông (D), Tây (T), Nam (N), Bắc (B). Viết chương trình điều khiển Robot đi theo câu lệnh là một chuỗi s gồm các ký tự D, T, N, B ($1 \leq \text{length}(s) \leq 10000$). Hãy in vị trí của Robot (tọa độ) sau mỗi bước đi.

**Input**

- Dòng thứ nhất chứa hai số nguyên X_s, Y_s là tọa độ ban đầu của Robot.
- Dòng thứ hai chứa câu lệnh s .

Output

- Nhiều dòng, mỗi dòng lần lượt là vị trí của Robot sau mỗi bước đi.

Ví dụ

Direction.txt	Moves.txt
1 1	1 1
DDDBB	2 1
	3 1
	4 1
	4 2
	4 3

BUỔI 02 - 03

KỸ THUẬT ĐỆ QUY

MỤC TIÊU

- Hiểu được cấu trúc của một hàm Đệ quy
- Vận dụng giải một số bài tập

Bài tập 1. Tính giai thừa

Cho số nguyên n ($0 \leq n \leq 12$). Viết hàm đệ quy tính $n!$ theo công thức đệ quy sau:

$$n! = \begin{cases} 1, & \text{nếu } n \leq 1 \\ n \times (n-1)!, & \text{nếu } n > 1 \end{cases}$$

Input

- Dòng duy nhất chứa số nguyên n

Output

- Giá trị của $n!$

Ví dụ

INPUT	OUTPUT
6	720

Bài tập 2. Tính lũy thừa

Cho hai số nguyên a, n ($a > 0, n \geq 0$). Viết hàm đệ quy tính a^n theo công thức đệ quy sau:

$$a^n = \begin{cases} 1, & \text{nếu } n = 0 \\ a^{\frac{n}{2}} \times a^{\frac{n}{2}}, & \text{nếu } n \text{ chẵn} \\ a \times a^{\frac{n-1}{2}} \times a^{\frac{n-1}{2}}, & \text{nếu } n \text{ lẻ} \end{cases}$$

Input

- Dòng duy nhất chứa hai số nguyên a và n

Output

- Giá trị của a^n

Ví dụ:

INPUT	OUTPUT
2 6	64

Bài tập 3. Tìm ước số chung lớn nhất bằng thuật toán Euclid

Cài đặt hàm đệ quy sử dụng thuật toán Euclid để tìm Ước số chung lớn nhất (USCLN) của 2 số nguyên dương m và n . Biết rằng:

$$USCLN(m, n) = \begin{cases} m, & \text{nếu } n = 0 \\ USCLN(n, m \% n), & \text{nếu } n > 0 \end{cases}$$

Input

- Dòng duy nhất chứa hai số nguyên dương m và n

Output

- USCLN của m và n

Ví dụ:

INPUT	OUTPUT
15 9	3

Bài tập 4. Chuyển đổi số từ hệ thập phân (DEC) sang hệ nhị phân (BIN)

Cho một số nguyên dương n . Viết hàm đệ quy chuyển đổi một số nguyên dương n từ hệ thập phân (DEC) sang hệ nhị phân (BIN).

2	20	 0
2	10	 0
2	5	 1
2	2	 0
	1	

Input

- Một dòng duy nhất chứa số nguyên dương n

Output

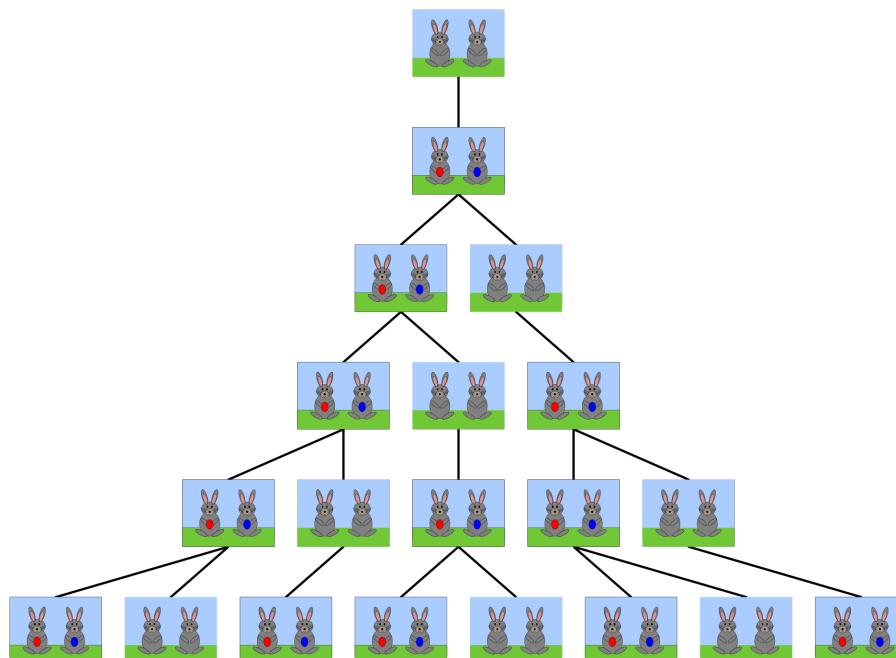
- Một dòng duy nhất chứa kết quả chuyển số nguyên dương n từ hệ DEC sang hệ BIN

Ví dụ:

INPUT	OUTPUT
20	10100

Bài tập 5. Bài toán số đôi thỏ

Một đôi thỏ (gồm một thỏ đực và một thỏ cái) không sinh cho đến khi chúng đủ 2 tháng tuổi. Sau khi đủ 2 tháng tuổi, mỗi đôi thỏ sinh một đôi thỏ con (gồm một thỏ đực và một thỏ cái) mỗi tháng. Hỏi sau n tháng có bao nhiêu đôi thỏ, tháng đầu tiên có một đôi thỏ sơ sinh:



Trong hình vẽ trên, ta quy ước:

- Cặp thỏ nâu là cặp thỏ có độ tuổi 1 tháng.
- Cặp thỏ được đánh dấu (màu đỏ và màu xanh) là cặp thỏ có khả năng sinh sản.

Nhìn vào hình vẽ trên ta nhận thấy:

- **Tháng đầu tiên và tháng thứ 2:** Chỉ có 1 đôi thỏ.
- **Tháng thứ ba:** đôi thỏ này sẽ đẻ ra một đôi thỏ con, do đó trong tháng này có 2 đôi thỏ.
- **Tháng thứ tư:** chỉ có đôi thỏ ban đầu sinh con nên đến thời điểm này có 3 đôi thỏ.
- **Tháng thứ năm:** có hai đôi thỏ (đôi thỏ đầu và đôi thỏ được sinh ra ở tháng thứ ba) cùng sinh con nên ở tháng này có $2 + 3 = 5$ đôi thỏ.
- **Tháng thứ sáu:** có ba đôi thỏ (2 đôi thỏ đầu và đôi thỏ được sinh ra ở tháng thứ tư) cùng sinh con ở thời điểm này nên đến đây có $3 + 5 = 8$ đôi thỏ.

Input

- Một dòng duy nhất chứa số nguyên dương n

Output

- Số đôi thỏ ở tháng thứ n

Ví dụ:

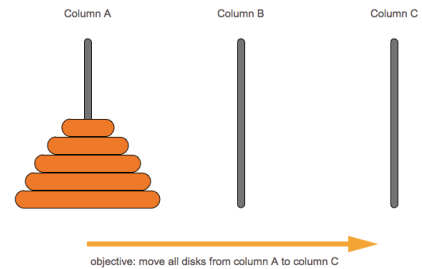
INPUT	OUTPUT
6	8

Bài tập 6. Trò chơi Tháp Hà Nội

Có 3 chiếc cột đặt tên là A, B và C. Ban đầu, đặt n cái đĩa có kích thước khác nhau ở cột A theo thứ tự kích thước các đĩa nhỏ dần từ đáy đến đỉnh cột (đĩa nhỏ nhất nằm trên cùng – xem hình bên)

Yêu cầu: Di chuyển n đĩa từ cột A sang cột C với luật chơi như sau:

- **Mỗi lần chỉ được di chuyển 1 đĩa nằm trên đỉnh** của một cột nào đó sang 1 cột bất kỳ trong 2 cột còn lại
- **Một đĩa chỉ có thể đặt lên phía trên đĩa có kích thước lớn hơn nó** (không nhất thiết hai đĩa này phải có kích thước liền kề)

**Input:**

- Một dòng duy nhất chứa số nguyên n – số đĩa ban đầu ở cột A

Output:

- Gồm nhiều dòng, mô tả các bước di chuyển đĩa, theo định dạng:

Chuyển 1 đĩa từ cột X -> Y

Trong đó: X, Y là tên cột (A/B/C)

Ví dụ:

INPUT	OUTPUT
2	Chuyển 1 đĩa từ cột A -> B Chuyển 1 đĩa từ cột A -> C Chuyển 1 đĩa từ cột B -> C

BUỔI 04 - 05

OOP: TÍNH ĐÓNG GÓI

MỤC TIÊU

- Hiểu được tính Đóng gói trong Lập trình hướng đối tượng
- Nắm vững cú pháp tạo và định nghĩa class trong C#

Bài tập 1. Class **Student**

Xây dựng class **Student** để biểu diễn các đối tượng sinh viên theo bản mô tả sau:

Student
- studentId: string - name: string - birthYear: int - gender: bool - stdClass: string
+ Student() + Student(string studentId, string name, int birthYear, bool gender, string stdClass) + GetStudentId(): string + SetStudentId (string newId): void + GetName():string + SetName(string newName): void + GetBirthYear(): int + SetBirthYear(int newBirthYear): void + GetGender(): bool + SetGender(bool newGender): void + GetStdClass(): string + SetStdClass(int newStdClass): void + Input(): void + GetAge(): int + PrintInfo(): void

- **Attributes:**
 - studentId: string
 - name: string
 - birthYear: int
 - gender: bool
 - stdClass: string
- **Constructors:**
 - Default constructor tạo một sinh viên không có các thông tin ban đầu: **Student()**
 - Khởi tạo một sinh viên với đầy đủ các thông tin ban đầu (**birthYear** cần được kiểm tra tính hợp lệ $1900 \leq \text{birthYear} \leq \text{năm hiện tại}$, nếu không hợp lệ thì thông báo và set về giá trị mặc định là 1900):
Student(string studentId, string name, int birthYear, bool gender, string stdClass)

Methods:

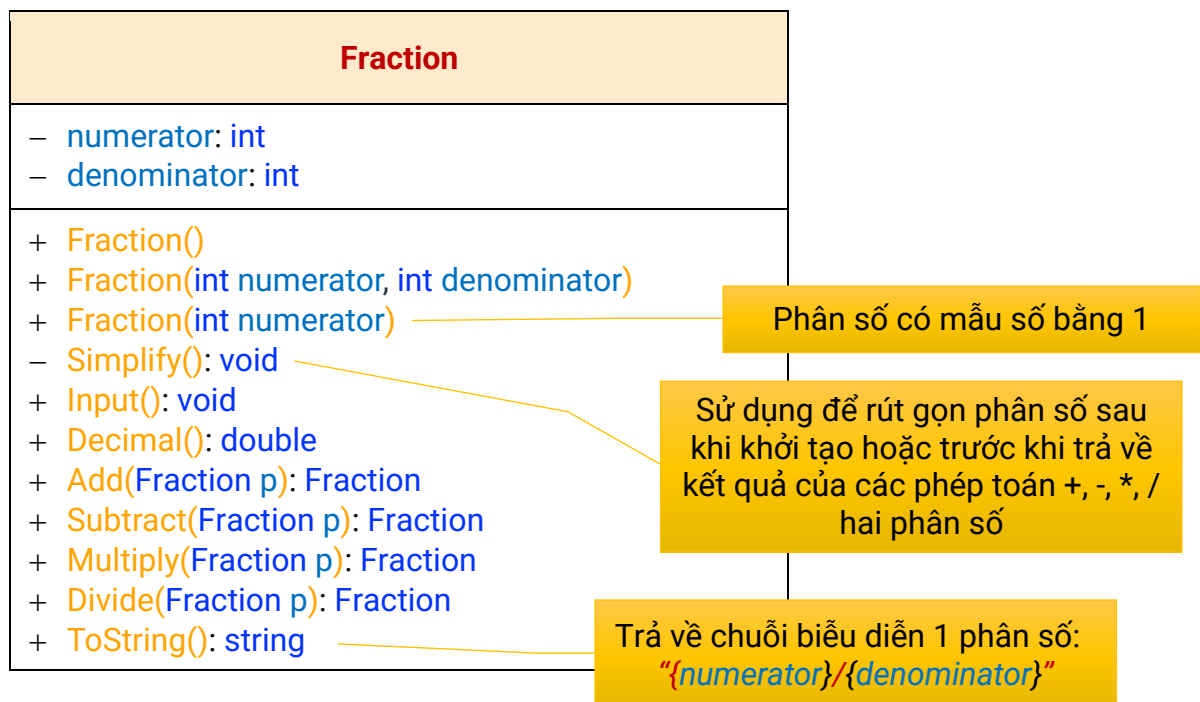
- Các phương thức **Get** và **Set** để truy xuất và thiết lập giá trị cho các thuộc tính của sinh viên. Đối với phương thức **Set** thuộc tính năm sinh cần được kiểm tra tính hợp lệ $1900 \leq \text{birthYear} \leq \text{năm hiện tại}$, nếu không hợp lệ thì thông báo và set về giá trị mặc định là 1900)
- **Input()**: Thực hiện nhập các thông tin cho sinh viên. Với năm sinh yêu cầu người dùng nhập cho đến thỏa điều kiện $1900 \leq \text{birthYear} \leq \text{năm hiện tại}$.
- **GetAge()**: Trả về tuổi hiện tại của sinh viên.
- **PrintInfo()**: In ra tất cả thông tin của sinh viên:

```
Student ID: { studentId }
Name: { name }
Birth Year: { birthYear }
Gender: { Nam/Nữ }
Class: { stdClass }
```

Viết hàm Main để khởi tạo các đối tượng sinh viên và thực hiện các phương thức đã xây dựng.

Bài tập 2. Class **Fraction**

Xây dựng class **Fraction** để biểu diễn phân số với các thuộc tính **numerator** (tử số) và **denominator** (mẫu số) nguyên.



- **Attributes:**
 - `numerator`: `int`
 - `denominator`: `int`
- **Properties:**
 - **Numerator**: bao gồm thao tác `get` truy xuất tử số và `set` để thiết lập giá trị cho tử số
 - **Denominator**: bao gồm thao tác `get` truy xuất mẫu số và `set` để thiết lập giá trị cho mẫu số (kiểm tra nếu giá trị mẫu số = 0 thì thông báo cho người dùng và set giá trị mặc định cho mẫu số 1)
- **Constructors:**
 - Default constructor tạo phân số có tử số = 0, mẫu số = 1: `Fraction()`
 - Khởi tạo phân số với giá trị tử số và mẫu số cho trước (*giá trị mẫu số được kiểm tra như phía trên*):
`Fraction(int numerator, int denominator)`
 - Khởi tạo phân số với giá trị tử số cho trước, mẫu số = 1:
`Fraction(int numerator)`

Yêu cầu: Các phân số sau khi được khởi tạo đều phải được rút gọn
- **Methods:**
 - `Simplify()`: Tối giản phân số
 - `Input()`: Thực hiện nhập tử số và mẫu số cho phân số. Với mẫu số yêu cầu người dùng nhập cho đến khi mẫu số khác 0. Phân số sau khi nhập xong sẽ được rút gọn.
 - `Decimal()`: Trả về giá trị thập phân của phân số
 - `Add(Fraction p)`: Trả về một phân số là tổng của đối tượng phân số hiện hành với một phân số p. Phân số kết quả phải được rút gọn trước khi trả về.
 - `Subtract(Fraction p)`: Trả về một phân số là hiệu của đối tượng phân số hiện hành với một phân số p. Phân số kết quả phải được rút gọn trước khi trả về.
 - `Multiply(Fraction p)`: Trả về một phân số là tích của đối tượng phân số hiện hành với một phân số p. Phân số kết quả phải được rút gọn trước khi trả về.
 - `Divide(Fraction p)`: Trả về một phân số là thương của đối tượng phân số hiện hành với một phân số p. Phân số kết quả phải được rút gọn trước khi trả về.
 - `ToString()`: Trả về chuỗi biểu diễn phân số dưới dạng: `"numerator/denominator"`

Viết hàm Main để khởi tạo các đối tượng phân số và thực hiện các phép toán trên các đối tượng này.

Bài tập 3. Class **Rectangle**

Xây dựng class **Rectangle** để biểu diễn các đối tượng hình chữ nhật theo bản mô tả sau:

(Sinh viên suy nghĩ các ràng buộc và các thuộc tính/phương thức cần thiết để mô tả được đối tượng đúng với thực tế. Ví dụ: chiều dài, chiều rộng của hình chữ nhật phải lớn hơn 0)

Rectangle
– width: double – height: double
+ Input(): void + ToString(): string + GetPerimeter(): double + GetArea(): double + IsSameArea(Rectangle rect): bool

Bài tập 4. Class **Date**

Xây dựng class **Date** để biểu diễn ngày tháng theo bản mô tả sau:

Date
– day: int – month: int – year: int
+ Date(day: int, month: int, year: int) + getDay(): int + getMonth(): int + getYear(): int + setDay(day: int): void + setMonth(month: int): void + setYear(year: int): void + setDate(day: int, month: int, year: int): void + toString(): String

day = [1, 31]
month = [1, 12]
year = [1900, 9999]
No input validation needed.

"dd/mm/yyyy" with leading zero

Bài tập 5. Class **Book**

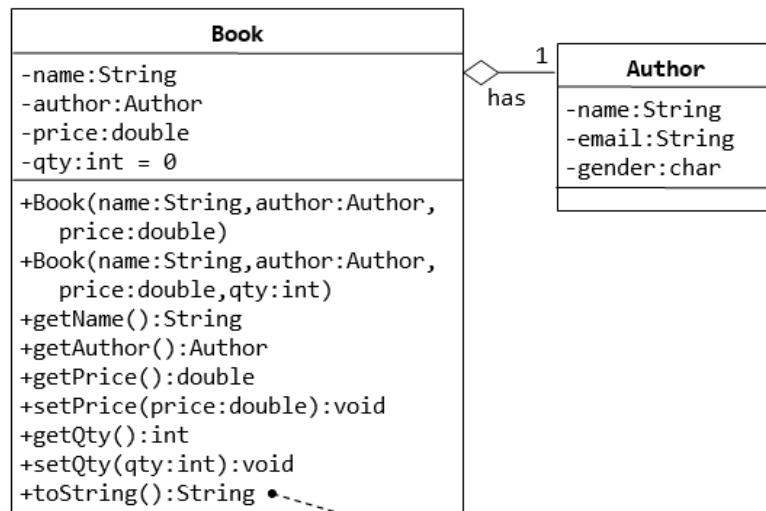
Xây dựng class **Author** để biểu diễn tác giả và class **Book** để biểu diễn quyển sách theo mô tả sau:

Author
– name: String – email: String – gender: char
+ Author(name: String, email: String, gender: char) + getName(): String + getEmail(): String + setEmail(email: String): void + getGender(): char + toString(): String

No default values for the variables

char of 'm' or 'f'

"Author[name=?, email=?, gender=?]"



"Book[name=?,Author[name=?,email=?,gender=?],price=?,qty=?]"
 You need to reuse Author's toString().

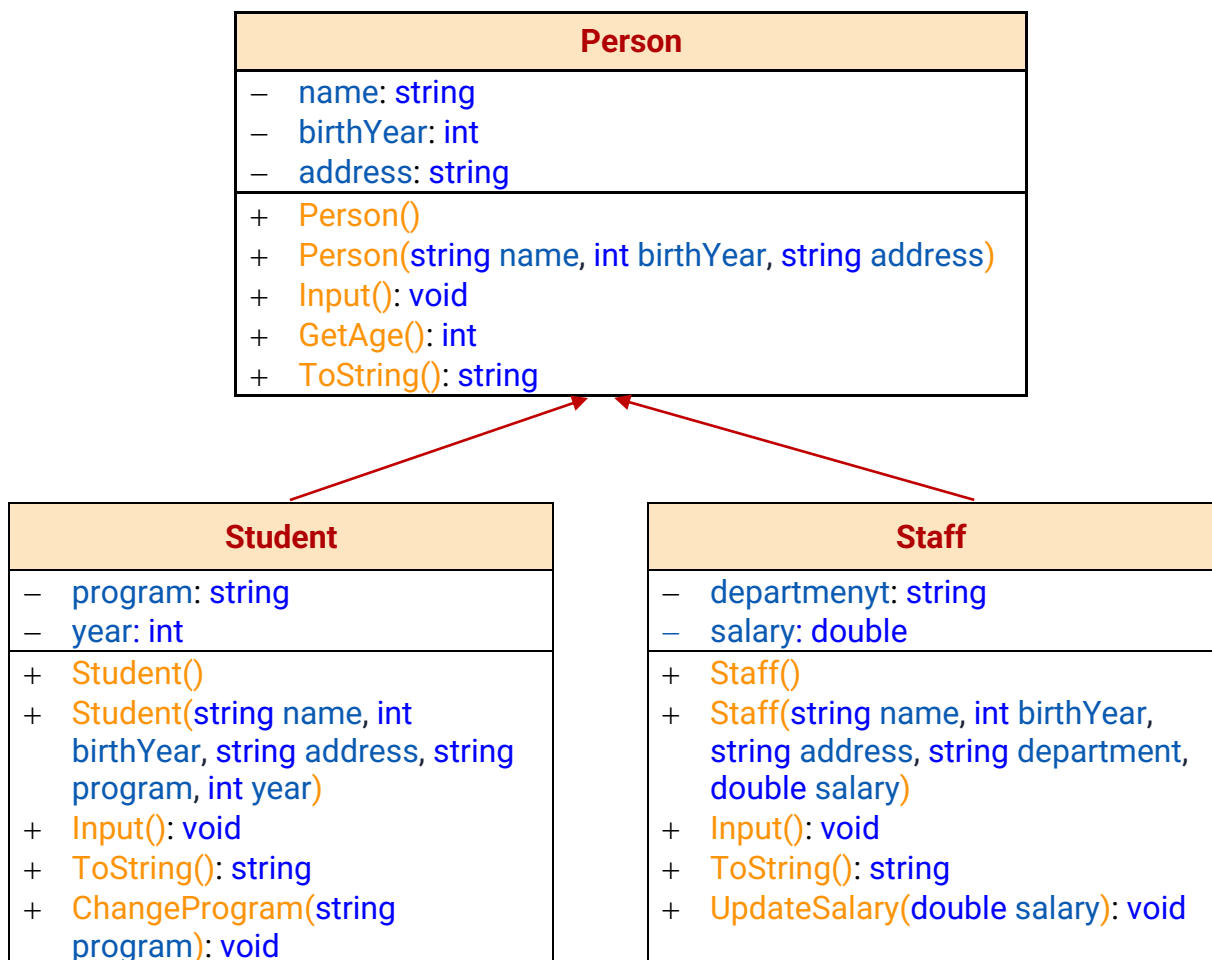
BUỔI 06

OOP: TÍNH KẾ THỪA VÀ ĐA HÌNH (PHẦN 1)

MỤC TIÊU

- Hiểu được tính Kế thừa và Đa hình trong Lập trình hướng đối tượng
- Nắm vững cú pháp tạo định nghĩa lớp dẫn xuất và ghi đè các phương thức (method overriding) của lớp cơ sở

Bài tập 1. Class **Person**, **Student** và **Staff**

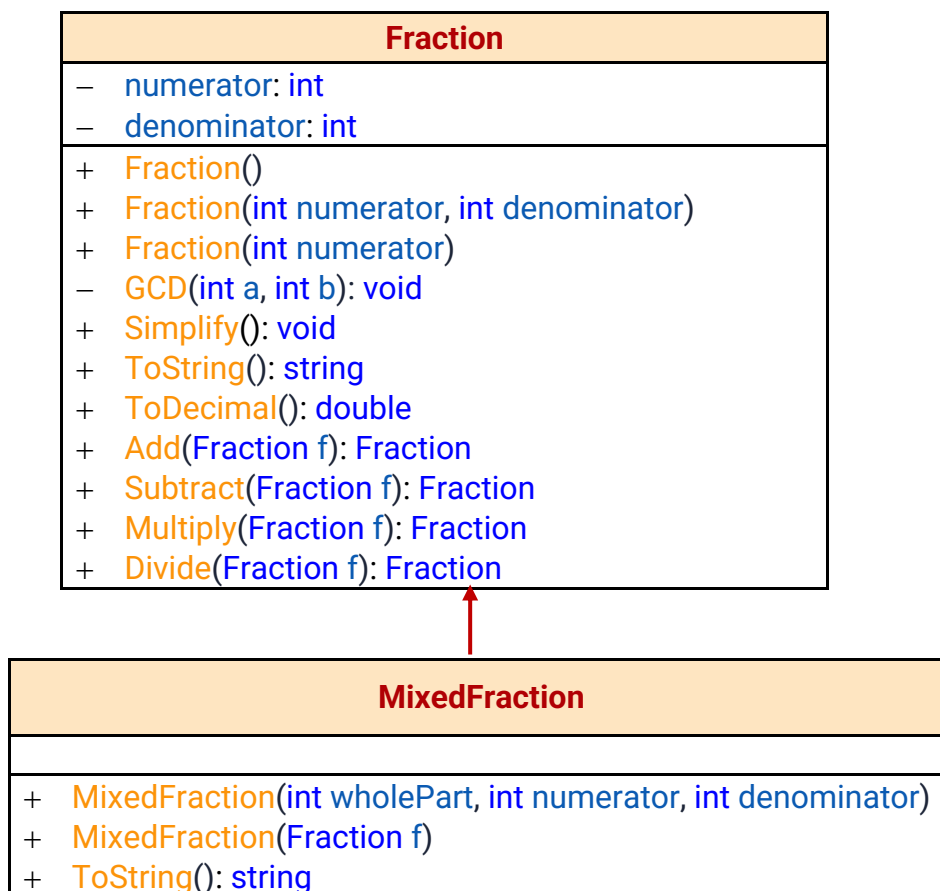


Mô tả:

- Class **Person**
 - + **Person**(string name, int birthYear, string address): Khởi tạo một đối tượng **Person** với các thông tin cho trước
 - + **Input**(): Phương thức để người dùng lần lượt nhập các thông tin cho một **Person**
 - + **GetAge**(): Tính tuổi của **Person** tính đến năm hiện tại
 - + **ToString**(): Trả về chuỗi biểu diễn một **Person**
"Person[Name: {name }, Age: {birthYear}, Address: {address}]"

- Class **Student**
 - + **Student**(string name, int birthYear, string address, string program, int year): Khởi tạo một đối tượng **Student** với các thông tin cho trước (gọi Constructor của class **Person**)
 - + **Input**(): Overriding phương thức **Input**() của class **Person** để người dùng nhập thêm các thông tin **program** và **year**
 - + **ToString**(): Overriding phương thức **ToString**() của class **Person** để trả về chuỗi biểu diễn một **Student**
"Student[Person[Name: {name}, Age: {birthYear}, Address: {address}] - Program: {program}, Year: {year}]"
- Class **Staff**
 - + **Staff**(string name, int birthYear, string address, string department, double salary): Khởi tạo một đối tượng **Staff** với các thông tin cho trước (gọi Constructor của class **Person**)
 - + **Input**(): Overriding phương thức **Input**() của class **Person** để người dùng nhập thêm các thông tin **department** và **salary**
 - + **ToString**(): Overriding phương thức **ToString**() của class **Person** để trả về chuỗi biểu diễn một **Staff**
"Staff[Person[Name: {name}, Age: {birthYear}, Address: {address}] - Department: {department}, Salary: {salary}]"

Bài tập 2. Class **Fraction** và **MixedFraction**



Mô tả:

- Class **Fraction**
 - + **Fraction**() : Khởi tạo phân số có tử số = 0, mẫu số = 1
 - + **Fraction**(int numerator, int denominator) : Khởi tạo phân số có tử số = numerator, mẫu số = denominator
 - + **Fraction**(int numerator) : Khởi tạo phân số có tử số = numerator, mẫu số = 1
 - + **ToString**() : Trả về chuỗi biểu diễn một phân số có dạng “*tử số/mẫu số*”
- Class **MixedFraction**
 - + **MixedFraction**(int wholePart, int numerator, int denominator) : Khởi tạo hỗn số có phần nguyên = wholePart và phần phân số có tử số = numerator, mẫu số = denominator

Gợi ý: Để việc xử lý đơn giản, các hỗn số đều được chuyển về lưu trữ dưới dạng phân số. Do đó constructor sẽ chuyển đổi hỗn số thành phân số và chỉ lưu trữ 2 thuộc tính numerator và denominator mà không cần khai báo thêm thuộc tính wholePart

- + **MixedFraction**(**Fraction** f) : Khởi tạo hỗn số từ một phân số
- + **ToString**() : Trả về chuỗi biểu diễn một hỗn số có dạng “*phần nguyên [tử số/mẫu số]*”

BUỔI 07

OOP: TÍNH KẾ THỪA VÀ ĐA HÌNH (PHẦN 2)

MỤC TIÊU

- Hiểu được tính Kế thừa và Đa hình trong Lập trình hướng đối tượng
- Nắm vững cú pháp nạp chồng phương thức (method overloading) và nạp chồng toán tử (operator overloading)
- Hiểu khái niệm thành viên tĩnh và sử dụng từ khóa “static

Bài tập 1. Class **Caculator**

Định nghĩa class **Caculator** theo mô tả dưới đây:

Caculator	
+	Caculator ()
+	Add (int a, int b): int
+	Add (double a, double b): double
+	Subtract (int a, int b): int
+	Subtract (double a, double b): double
+	Multiply (int a, int b): int
+	Multiply (double a, double b): double
+	Divide (int a, int b): double
+	Divide (double a, double b): double
+	Min (int a, int b): int
+	Min (int a, int b, int c): int
+	Min (double a, double , double c): double
+	Max (int a, int b): int
+	Max (int a, int b, int c): int
+	Max (double a, double , double c): double

- Viết hàm Main(), tạo đối tượng của class **Caculator** và gọi các phương thức tương ứng
- Chuyển tất cả phương thức thành phương thức tĩnh (**static**), nhận xét sự khác nhau giữa **static method** và **non-static method**

Bài tập 2. Class **Fraction** và **MixedFraction**

Thực hiện nạp chồng các toán tử (operator overloading) cho class **Fraction** và class **MixedFraction** đã định nghĩa ở Buổi 07

- Nạp chồng toán tử một ngôi “-” để đổi dấu phân số/hỗn số
- Nạp chồng các toán tử hai ngôi “+”, “-”, “*”, “/” để thực hiện các phép toán giữa 2 phân số/hỗn số
- Nạp chồng các toán tử so sánh 2 phân số/hỗn số “==”, “!=”, “>”, “<”, “>=”, “<=”

BUỔI 08

OOP: LỚP TRỪU TƯỢNG

MỤC TIÊU

- Hiểu khái niệm abstract class, abstract method và cài đặt trong C#

Bài tập 1. Class **Shape**, **Circle**, **Rectangle** và **Square**



Bài tập 2. Class **Fraction**

Hãy định nghĩa các phương thức Add, Subtract, Multiply, Divide ở lớp Fraction thành phương thức tĩnh (static method)

--- HẾT ---

